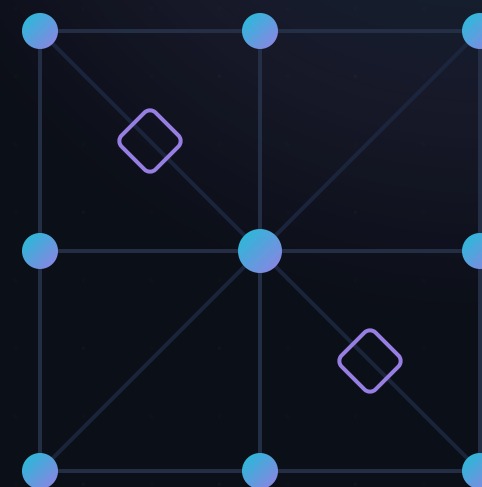


SEA QUANTATHON 2026

Quantum Learning-Based Methods for Fault-Tolerant Quantum Computation

Learning-based decoders & state preparation for quantum error correction

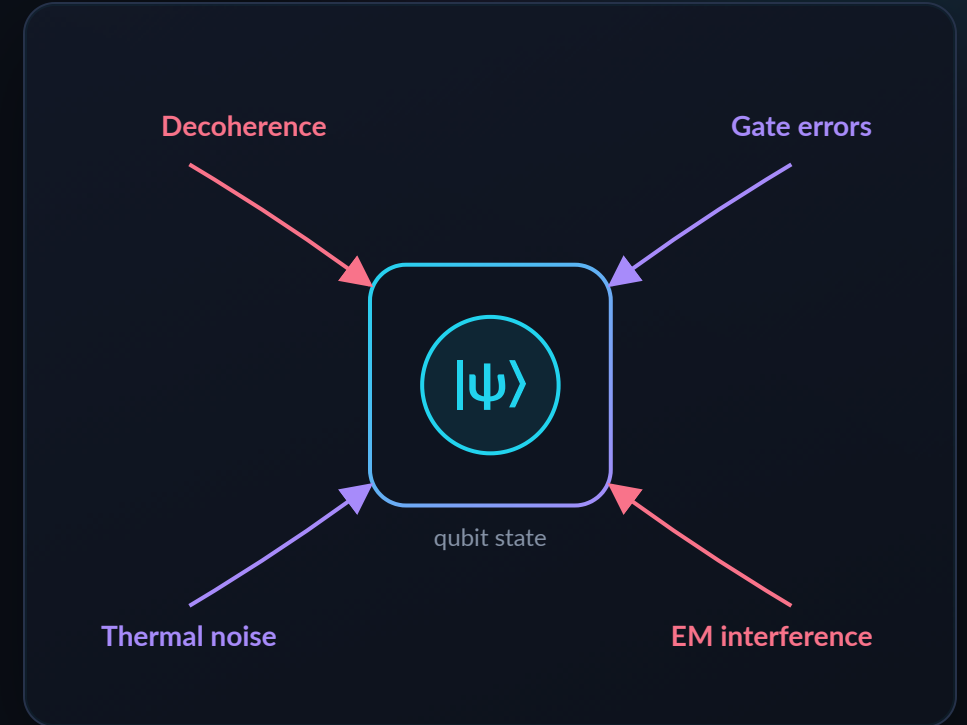
Chu Tuan Kiet · Nguyen Gia Hien · Hoang Dinh Duy Anh · Truong Le Gia Khanh



Where quantum errors come from

In quantum systems, errors arise from undesired interactions with the environment (e.g., decoherence) or imperfect control operations (e.g., gate inaccuracies).

⇒ Leads to the accumulation and propagation of errors throughout the computation process.



A quantum system interacts with its environment, so it suffers from noise.

Fault-tolerant quantum computation

- ▶ **Fault-tolerant quantum computation:** design circuits and error-correction codes that tolerate a certain level of errors without failing.
- ▶ However, optimizing fault-tolerance efficiency is highly challenging – we must balance the resources spent on error correction.



One example of a fault-tolerant quantum computation scheme.

What we need to solve

- ▶ Prepare fault-tolerant, high-fidelity error-correction auxiliary states.
- ▶ Generalize to various noise environments and hardware (e.g. cat states).
- ▶ A scalable fault-tolerant scheme.



High-fidelity aux states
prepare & verify fault-tolerantly



Generalize: noise & hardware
robust across devices, cat states



Scalable FT scheme
grow qubits without runaway cost

The problem of the proposal, in three parts.

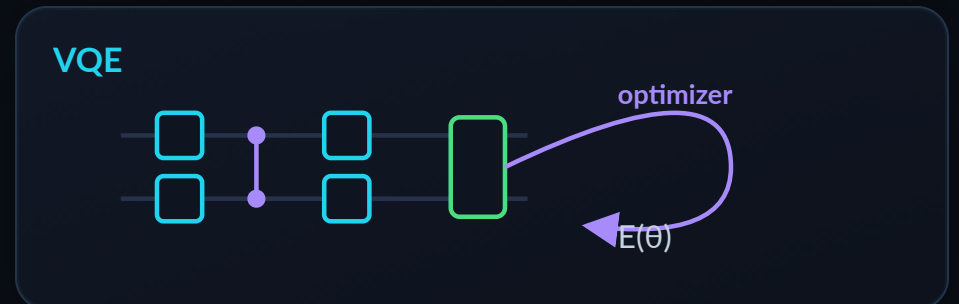
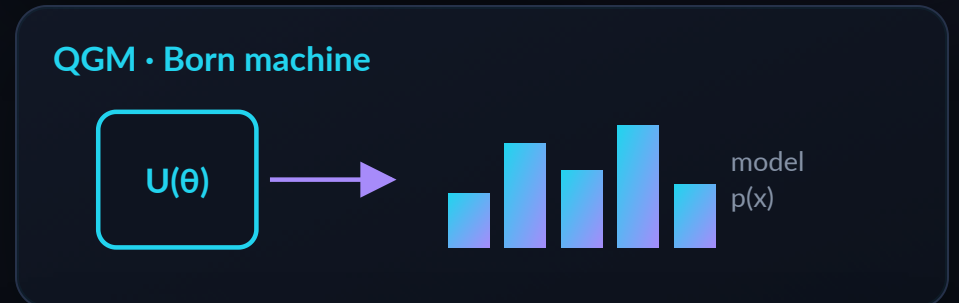
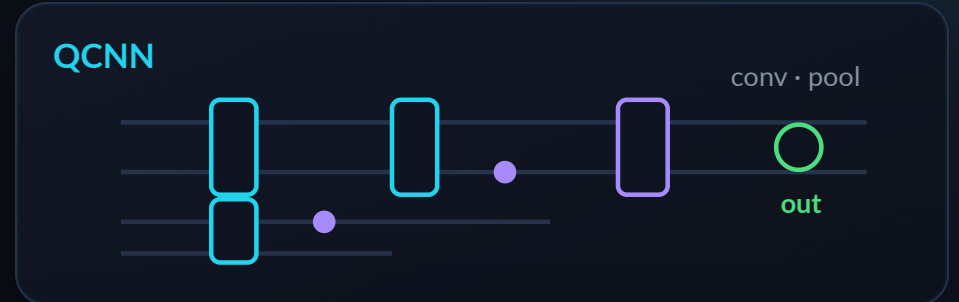
A staged quantum-learning program

A three-year program applying quantum learning-based methods, rolled out in stages — **QCNN** → **VQE** → **QGM** — to:

- ▶ Accelerate and automate preparation of fault-tolerant, high-fidelity error-correction auxiliary states.
- ▶ Adapt to diverse noise profiles under different hardware constraints.
- ▶ Minimize qubit count so the architecture stays scalable.

TODAY'S DEMO

Illustrates the QCNN start-up phase, applied to a concrete QEC task (syndrome **decoding**) before expanding to auxiliary-state preparation in later phases.



Why quantum learning-based methods

- ▶ Conventional QEC codes (e.g. surface, topological) don't scale well with qubit count $\Rightarrow$ may introduce more noise to the system.
- ▶ Machine-learning methods often need **quantum-classical** communication, adding delays $\Rightarrow$ decreases coherence time.
- ▶ Quantum learning-based methods act **directly on the quantum states** $\rightarrow$ can adapt to various noise characteristics.

Unscalable & noisy

noise grows with qubit count



Delayed communication

quantum-classical round-trips



Adaptive QML

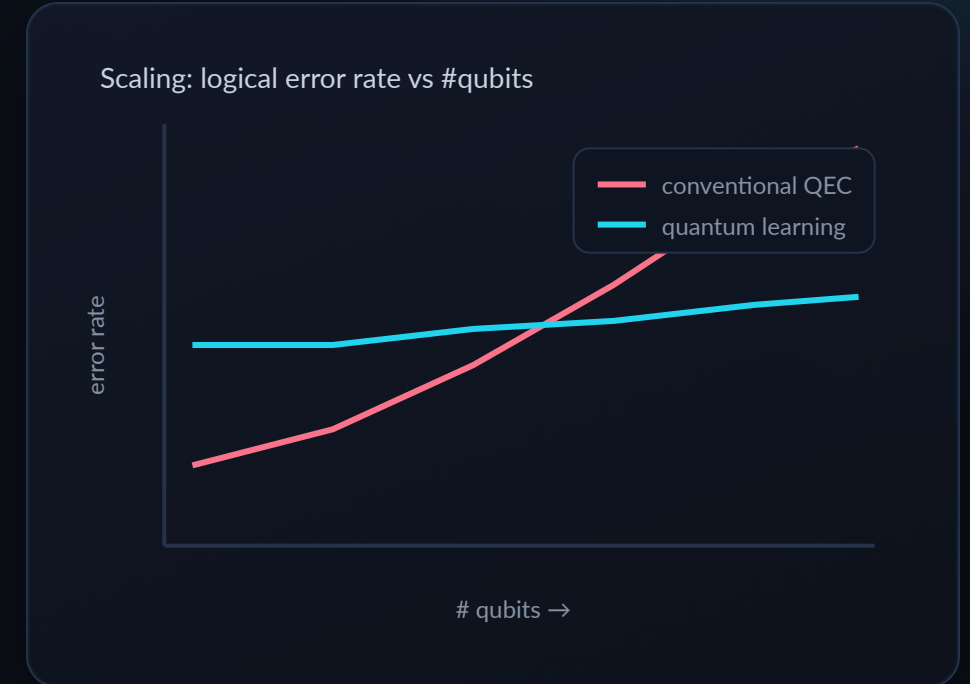
acts directly on quantum states



Why we use quantum in this project.

What each method should deliver

- ▶ **QCNN:** barren-plateau-free, scales with qubit count better than previous quantum ML methods.
Evidence in this demo: a QCNN syndrome decoder benchmarked against an MWPM baseline on the surface code, measured by **logical error rate** and **threshold**.
- ▶ **VQE:** adaptive and robust to noise, improved accuracy.
- ▶ **QGM:** generalizes across noise characteristics, efficiently scalable.



Expected: QML stays flat/better as qubits grow.

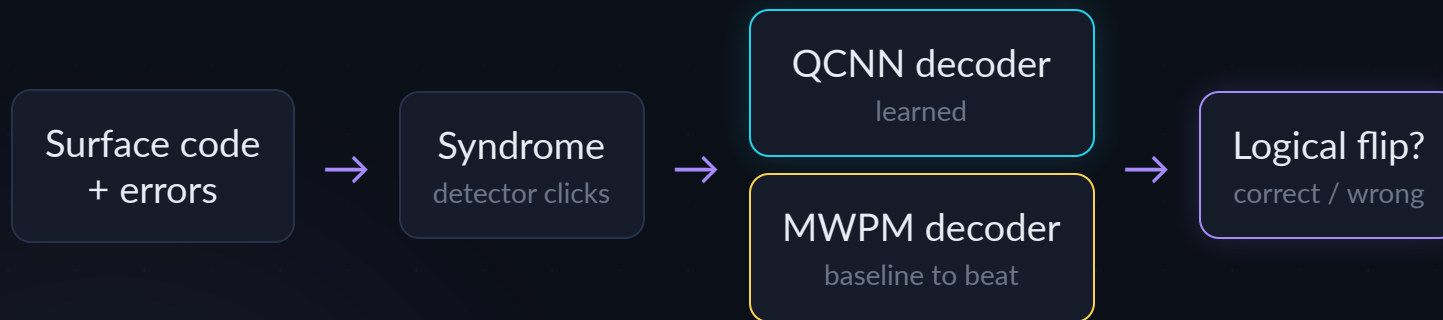
A narrow proof-of-concept

This hackathon deliverable is a **narrow proof-of-concept**: it demonstrates QCNN capability on **one** concrete QEC task — syndrome **decoding** of the surface code — and is **not** the full auxiliary-state-preparation roadmap.

- ▶ Proof-of-concept scale: start at a single small code distance, one code family, before scaling up.
- ▶ Validates the **QCNN building block** the whole three-year program is built on.
- ▶ Auxiliary-state preparation, VQE and QGM remain **future phases** — see the Roadmap slides.

Decoding the surface code

- ▶ **Task:** read the error “syndrome” from a surface-code patch and predict whether the logical qubit flipped.
- ▶ **Surface code, distance d:** more physical qubits give stronger protection (we start small).
- ▶ **Syndrome:** the pattern of detector “clicks” from repeated stabilizer measurements — the only thing a decoder sees.
- ▶ **Head-to-head:** a learned **QCNN** decoder vs the standard **MWPM** baseline, same data, same metric.



QCNN vs MWPM

Logical error rate

QCNN vs MWPM, per distance

[SỐ LIỆU SẼ CẬP NHẬT SAU KHI
CHẠY XONG BACKEND]

Threshold plot

logical error rate vs physical error rate

[THRESHOLD PLOT – SẼ CẬP NHẬT
SAU KHI CHẠY XONG BACKEND]

Inference latency

per-shot, QCNN vs MWPM

[SỐ LIỆU SẼ CẬP NHẬT SAU KHI
CHẠY XONG BACKEND]

2026 – 2027

June 2026 – December 2026

- ▶ **June:** read background, reproduce results of previous research.
- ▶ **July:** build a QCNN-based architecture and test it on several benchmarks and one hardware platform.
- ▶ **Aug–Oct:** adapt the QCNN architecture to different hardware platforms and noise characteristics.
- ▶ **Nov–Dec:** improve the QCNN method in preparation for publication.

January 2027 – June 2027

- ▶ **Jan:** wrap up QCNN work as an open-source package.
- ▶ **Feb:** read background on VQE.
- ▶ **Mar–May:** build a VQE-based architecture, evaluate across multiple benchmarks and hardware types.
- ▶ **June:** improve VQE methods for best results on the problem.

2027 — 2028

June 2027 — December 2027

- ▶ **June:** wrap up VQE work as an open-source package.
- ▶ **July:** read background on QGM.
- ▶ **Aug–Nov:** build a QGM-based architecture, test on several benchmarks and various hardware platforms.
- ▶ **Nov–Dec:** improve QGM methods for best results, toward publication.

January 2028 — June 2028

- ▶ **Jan:** wrap up QGM work as an open-source package.
- ▶ **Feb–Mar:** build an agentic AI system across all three methods that automatically chooses parameters and methods.
- ▶ **Apr–June:** release the integrated framework as an open-source package and prepare final publications.